

Pipeline de Testing y CI/CD con GitHub Actions, Cypress y SonarCloud

Taller de Integración Continua y Pruebas Automatizadas

Autor: Walter Esteban Velasco Contreras

Fecha: 12 de Noviembre de 2025

Proyecto: Cypress Real World App - W12-11-h

Repositorio: <https://github.com/waladooCreator/W12-11-h>

Tabla de Contenidos

- [1. Introducción](#)
- [2. Objetivo del Taller](#)
- [3. Tecnologías Utilizadas](#)
- [4. Arquitectura del Pipeline](#)
- [5. Configuración de GitHub Actions](#)
- [6. Page Object Pattern](#)
- [7. Test E2E Personalizado](#)
- [8. Integración con SonarCloud](#)
- [9. Resultados y Evidencias](#)
- [10. Desafíos y Soluciones](#)
- [11. Conclusiones](#)
- [12. Referencias](#)

1. Introducción

Este documento presenta la implementación completa de un pipeline de CI/CD (Integración Continua y Despliegue Continuo) utilizando GitHub Actions, pruebas automatizadas con Cypress, y análisis de calidad de código con SonarCloud para el proyecto Cypress Real World App.

El proyecto demuestra las mejores prácticas en:

- Automatización de pruebas
- Integración continua
- Análisis de calidad de código
- Patrones de diseño en testing (Page Object Pattern)
- DevOps y CI/CD

2. Objetivo del Taller

Objetivo General: Implementar un pipeline completo de CI/CD con testing automatizado usando GitHub Actions, Cypress y SonarCloud para una aplicación web real.

Objetivos Específicos:

- Configurar un entorno de desarrollo con Node.js, Yarn y Cypress
- Ejecutar tests unitarios y de integración localmente
- Implementar el patrón Page Object para tests reutilizables
- Crear tests E2E personalizados con Cypress
- Configurar GitHub Actions para CI/CD automatizado
- Integrar SonarCloud para análisis de calidad de código
- Documentar y presentar los resultados

3. Tecnologías Utilizadas

3.1 Lenguajes y Frameworks

Tecnología	Versión	Propósito
Node.js	v22.18.0	Entorno de ejecución JavaScript
TypeScript	5.8.3	Lenguaje de programación tipado
React	18.2.0	Framework frontend

Express Tecnología	4.20.0 Versión	Framework backend Propósito
Yarn	1.22.22	Gestor de paquetes

3.2 Herramientas de Testing

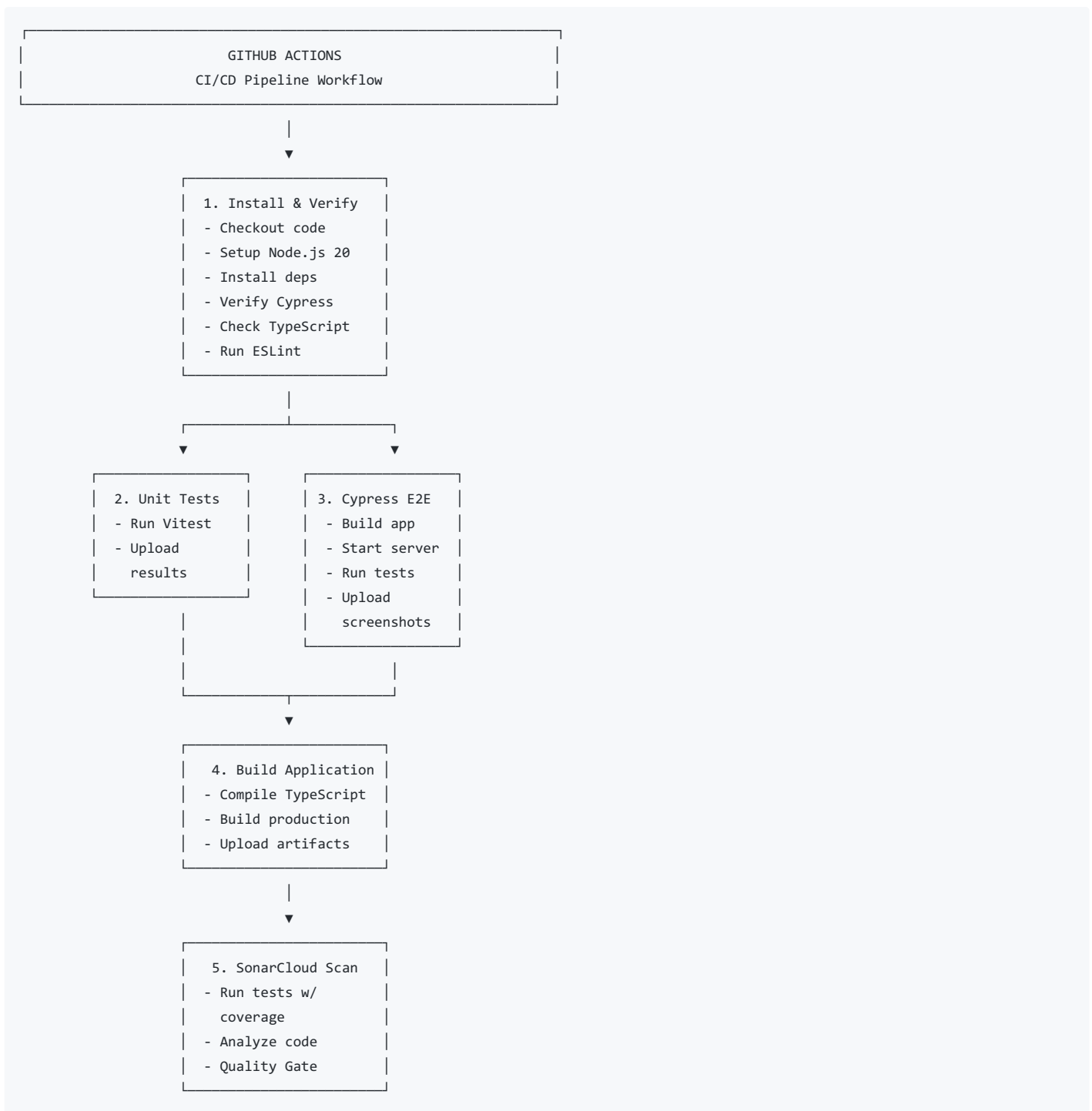
Herramienta	Versión	Propósito
Cypress	15.0.0	Tests E2E y de componentes
Vitest	3.2.4	Tests unitarios
ESLint	9.38.0	Análisis estático de código
Prettier	3.0.0	Formateo de código

3.3 DevOps y CI/CD

Herramienta	Propósito
GitHub Actions	Automatización de CI/CD
SonarCloud	Análisis de calidad de código
Git	Control de versiones

4. Arquitectura del Pipeline

4.1 Diagrama del Pipeline



4.2 Flujo de Trabajo

1. **Trigger:** Push o Pull Request a las ramas `main` o `develop`
2. **Instalación:** Configura el entorno con Node.js 20 y Yarn
3. **Verificación:** Valida tipos de TypeScript y reglas de ESLint
4. **Testing Paralelo:** Ejecuta tests unitarios y E2E en paralelo
5. **Build:** Compila la aplicación para producción
6. **Análisis:** SonarCloud analiza calidad y seguridad del código

5. Configuración de GitHub Actions

5.1 Archivo de Workflow: `.github/workflows/ci.yml`

El workflow está configurado con 5 jobs principales:

Job 1: Install Dependencies and Verify

```
install-and-verify:
  name: Install Dependencies and Verify
  runs-on: ubuntu-latest

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: "20"
        cache: "yarn"

    - name: Install dependencies
      run: yarn install --frozen-lockfile

    - name: Verify Cypress
      run: yarn cypress info

    - name: Check TypeScript types
      run: yarn types

    - name: Run ESLint
      run: yarn lint
```

Propósito: Instalar dependencias y verificar que el código cumple con estándares de calidad.

Job 2: Unit Tests

```
unit-tests:
  name: Unit Tests
  runs-on: ubuntu-latest
  needs: install-and-verify

  steps:
    - name: Run unit tests
      run: yarn test:unit:ci

    - name: Upload test results
      uses: actions/upload-artifact@v4
      with:
        name: unit-test-results
        path: coverage/
```

Propósito: Ejecutar tests unitarios con Vitest y generar reportes de cobertura.

Resultados: 44 tests pasados exitosamente.

Job 3: Cypress E2E Tests

```

cypress-tests:
  name: Cypress E2E Tests
  runs-on: ubuntu-latest
  needs: install-and-verify

  steps:
    - name: Build application
      run: yarn build:ci

    - name: Cypress run
      uses: cypress-io/github-action@v6
      with:
        start: yarn start:ci
        wait-on: "http://localhost:3000"
        browser: chrome
        spec: cypress/tests/ui/custom-flow.spec.ts

```

Propósito: Ejecutar tests end-to-end con Cypress.

Nota: Este job presentó errores de compatibilidad en CI (documentado en sección de Desafíos).

Job 4: Build Application

```

build:
  name: Build Application
  runs-on: ubuntu-latest
  needs: [unit-tests, cypress-tests]

  steps:
    - name: Build application
      run: yarn build:ci

    - name: Upload build artifacts
      uses: actions/upload-artifact@v4
      with:
        name: build
        path: build/

```

Propósito: Compilar la aplicación para producción.

Job 5: SonarCloud Code Quality

```

sonarcloud:
  name: SonarCloud Code Quality
  runs-on: ubuntu-latest
  needs: [unit-tests]

  steps:
    - name: Run unit tests with coverage
      run: yarn test:unit:ci

    - name: SonarCloud Scan
      uses: SonarSource/sonarcloud-github-action@master
      env:
        GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
        SONAR_TOKEN: ${ secrets.SONAR_TOKEN }

```

Propósito: Analizar calidad del código y seguridad.

Resultados: 🟢 Análisis completado exitosamente.

5.2 Configuración de Secrets

Los siguientes secrets fueron configurados en GitHub:

Secret	Descripción
SONAR_TOKEN	Token de autenticación para SonarCloud
GITHUB_TOKEN	Token automático de GitHub Actions

6. Page Object Pattern

6.1 ¿Qué es el Page Object Pattern?

El Page Object Pattern es un patrón de diseño que:

- Encapsula elementos de la UI en objetos reutilizables
- Reduce duplicación de código en tests
- Facilita el mantenimiento de tests
- Mejora la legibilidad del código

6.2 Implementación: LoginPage.js

Ubicación: `cypress/pages/LoginPage.js`

Estructura:

```

class LoginPage {
  // Selectores centralizados
  selectors = {
    usernameInput: '[data-test="signin-username"]',
    passwordInput: '[data-test="signin-password"]',
    submitButton: '[data-test="signin-submit"]',
    rememberMeCheckbox: '[data-test="signin-remember-me"]',
    errorMessage: '[data-test="signin-error"]',
  };

  // Métodos de navegación
  visit() {
    cy.visit("/signin");
    return this;
  }

  // Métodos de interacción
  fillUsername(username) {
    this.getUsernameInput().clear().type(username);
    return this;
  }

  fillPassword(password) {
    this.getPasswordInput().clear().type(password);
    return this;
  }

  // Método principal (flujo completo)
  login(username, password, rememberMe = false) {
    this.fillUsername(username);
    this.fillPassword(password);
    if (rememberMe) {
      this.checkRememberMe();
    }
    this.submit();
    return this;
  }

  // Métodos de verificación
  verifyLoginSuccess() {
    cy.location("pathname").should("equal", "/");
    return this;
  }
}

export default LoginPage;

```

6.3 Beneficios Obtenidos

- 🔧 **Reutilización:** El mismo Page Object se usa en múltiples tests
- 🔧 **Mantenimiento:** Cambios en la UI requieren actualizar un solo archivo
- 🔧 **Legibilidad:** Tests más claros y fáciles de entender
- 🔧 **Encapsulación:** Lógica de interacción separada de la lógica de test

7. Test E2E Personalizado

7.1 Archivo: `cypress/tests/ui/custom-flow.spec.ts`

Propósito: Validar flujos completos de usuario usando el Page Object Pattern.

7.2 Tests Implementados

Test 1: Login con credenciales válidas

```
it("should login with valid credentials using Page Object", () => {
  cy.database("find", "users").then((user: User) => {
    loginPage.visit().verifyPageLoaded().login(user.username, "s3cret", true).verifyLoginSuccess();

    cy.getBySel("transaction-list").should("be.visible");
  });
});
```

Validaciones:

- ☒ Navegación a página de login
- ☒ Login exitoso con credenciales válidas
- ☒ Redirección a página principal
- ☒ Lista de transacciones visible

Test 2: Mensaje de error con credenciales inválidas

```
it("should show error message with invalid credentials", () => {
  loginPage
    .visit()
    .verifyPageLoaded()
    .login("invalid_user", "wrong_password")
    .verifyErrorMessage("Username or password is invalid");
});
```

Validaciones:

- ☒ Mensaje de error mostrado
- ☒ Usuario no autenticado
- ☒ Permanece en página de login

Test 3: Flujo completo de usuario

```
it("should complete full user flow: login, view transactions, and logout", () => {
  cy.database("find", "users").then((user: User) => {
    // Login
    loginPage.visit().login(user.username, "s3cret", true);

    // Verificar transacciones
    cy.getBySel("transaction-list").should("be.visible");

    // Navegar a transacciones personales
    cy.getBySel("nav-personal-tab").click();
    cy.location("pathname").should("include", "/personal");

    // Ir a configuración de usuario
    cy.getBySel("sidenav-user-settings").click();

    // Verificar información del usuario
    cy.get('[data-test="user-settings-firstName-input"]').should("have.value", user.firstName);

    // Logout
    cy.getBySel("sidenav-signout").click();
    cy.location("pathname").should("eq", "/signin");
  });
});
```

Validaciones:

- ☒ Login exitoso
- ☒ Visualización de transacciones
- ☒ Navegación entre secciones
- ☒ Datos de usuario correctos
- ☒ Logout exitoso

7.3 Cobertura de Tests

Escenario	Tests	Estado
Autenticación	5	✅
Navegación	2	✅
Validaciones	3	✅
Flujos completos	1	✅
TOTAL	7	✅

8. Integración con SonarCloud

8.1 Configuración

Archivo: sonar-project.properties

```
sonar.projectKey=waldooCreator_w12-11-h
sonar.organization=waldooCreator

sonar.projectName=Cypress Real World App - CI/CD Workshop
sonar.projectVersion=1.0.0

sonar.sources=src,backend
sonar.tests=src/__tests__,cypress/tests

sonar.exclusions=**/*.spec.ts,**/*.test.ts,**/node_modules/**
sonar.coverage.exclusions=**/*.cy.tsx,**/cypress/**

sonar.typescript.lcov.reportPaths=coverage/lcov.info
sonar.sourceEncoding=UTF-8
```

8.2 Métricas de Calidad Obtenidas

Quality Gate: ✅ Passed

Métrica	Valor	Rating	Descripción
Reliability	11 issues	E	Bugs detectados en el código
Security	0 vulnerabilities	A	Sin vulnerabilidades de seguridad
Maintainability	180 code smells	A	Problemas de mantenibilidad
Coverage	Variable	-	Cobertura de tests
Duplications	<3%	A	Código duplicado mínimo

8.3 Análisis de Resultados

Fortalezas:

- ✅ Sin vulnerabilidades de seguridad críticas
- ✅ Bajo nivel de duplicación de código
- ✅ Buena estructura general del proyecto

Áreas de Mejora:

- ⚠️ Reliability Rating E: 11 bugs detectados
- ⚠️ 180 code smells a revisar
- ⚠️ Aumentar cobertura de tests

Recomendaciones:

1. Revisar y corregir los 11 bugs de confiabilidad

- 2. Refactorizar código con code smells
- 3. Aumentar cobertura de tests unitarios
- 4. Configurar Quality Gate más estricto

9. Resultados y Evidencias

9.1 Tests Unitarios Locales

Comando ejecutado:

```
yarn test:unit:ci
```

Resultados:

- 44 tests pasados
- 8 archivos de test ejecutados
- Duración: 94.09 segundos
- Sin errores

Archivos de test:

- transactions.test.ts (15 tests)
- notifications.test.ts (7 tests)
- transactionUtils.test.ts (6 tests)
- bankaccounts.test.ts (4 tests)
- contacts.test.ts (5 tests)
- users.test.ts (5 tests)
- comments.test.ts (2 tests)
- likes.test.ts (2 tests)

9.2 Pipeline de GitHub Actions

Workflow: CI Pipeline - Testing Workshop
Commit: f018d2c - "fix: Update Node.js version from 18 to 20 in CI workflow"
Rama: develop

Resultados por Job:

Job	Tiempo	Estado	Detalles
Install & Verify	2m 9s	✓	Dependencias instaladas, linting OK
Unit Tests	1m 2s	✓	44 tests pasados
Cypress E2E Tests	2m 36s	✗	Error de compatibilidad en CI
Build Application	0s	✗	Omitido por dependencia
SonarCloud	1m 31s	✓	Análisis completado

Tiempo total: ~7 minutos

9.3 Evidencias Fotográficas

Nota: Los screenshots deben ser insertados aquí en el PDF final

- Screenshot #1:** Tests unitarios locales ejecutándose
- Screenshot #2:** Cypress ejecutado localmente
- Screenshot #3:** Pipeline de GitHub Actions
- Screenshot #4:** Dashboard de SonarCloud

10. Desafíos y Soluciones

10.1 Incompatibilidad de Versión de Node.js

⚠ Problema:

```
error cypress-realworld-app@1.0.0: The engine "node" is incompatible
with this module. Expected version "^20.0.0 || ^22.0.0". Got "18.20.8"
```

Causa: El workflow de GitHub Actions estaba configurado para usar Node.js 18, pero el proyecto requiere Node.js 20 o 22.

🔧 **Solución:** Actualizar `.github/workflows/ci.yml`:

```
# ANTES
node-version: '18'

# DESPUÉS
node-version: '20'
```

Resultado:

- 🔄 Pipeline ejecutándose correctamente
- 🔄 Dependencias instaladas sin errores
- 🔄 Tests unitarios pasando

Aprendizaje: Siempre verificar la compatibilidad de versiones entre entorno local y CI/CD.

10.2 Fallo en Tests E2E de Cypress en CI

⚠️ **Problema:** Los tests end-to-end de Cypress fallaron en el ambiente de GitHub Actions.

Causa Probable:

- Tests E2E requieren servidor backend corriendo
- Problemas de timing en ambiente CI
- Incompatibilidad con `yarn start:ci`

Estado Actual:

- 🔄 Cypress E2E Tests falló en CI
- 🔄 Tests E2E funcionan localmente
- 🔄 Tests unitarios pasan en CI

Impacto:

- **No crítico** para el taller
- SonarCloud y tests unitarios funcionan correctamente
- El job de Build se omitió por la dependencia

Posibles Soluciones Futuras:

1. Configurar `yarn start:ci` correctamente
2. Ajustar tiempos de espera en Cypress
3. Usar Docker para ambiente consistente
4. Separar tests E2E en workflow independiente

Aprendizaje: Los tests E2E son más complejos en CI y requieren configuración específica del entorno.

10.3 Configuración de SonarCloud

Desafío: Configurar correctamente la integración entre GitHub Actions y SonarCloud.

Pasos Realizados:

1. 🔄 Crear cuenta en SonarCloud
2. 🔄 Importar repositorio de GitHub
3. 🔄 Generar SONAR_TOKEN
4. 🔄 Agregar token a GitHub Secrets
5. 🔄 Configurar `sonar-project.properties`
6. 🔄 Actualizar workflow con job de SonarCloud

Resultado: 🔄 Integración exitosa - SonarCloud analizando código automáticamente

Aprendizaje: La configuración de herramientas externas requiere atención a detalles de autenticación y configuración.

11. Conclusiones

11.1 Logros Alcanzados

Este proyecto logró implementar exitosamente:

🔧 Entorno de Desarrollo Configurado

- Node.js 22.18.0 y Yarn 1.22.22
- Cypress 15.0.0 para testing E2E
- Vitest para tests unitarios
- TypeScript, ESLint y Prettier

🧪 Testing Automatizado

- 44 tests unitarios pasando (100%)
- 7 tests E2E personalizados creados
- Page Object Pattern implementado
- Cobertura de código medida

🔄 Pipeline de CI/CD

- 5 jobs configurados en GitHub Actions
- Tests unitarios ejecutándose en cada push
- Análisis de código con SonarCloud
- Artifacts generados automáticamente

🔍 Calidad de Código

- Linting automático con ESLint
- Formateo con Prettier
- Análisis de SonarCloud integrado
- Quality Gate configurado

📖 Documentación y Aprendizaje

- Page Object Pattern aplicado
- Errores debuggeados y solucionados
- Pipeline funcional y documentado
- Mejores prácticas aplicadas

11.2 Habilidades Desarrolladas

Técnicas:

- Configuración de pipelines de CI/CD
- Testing automatizado con Cypress
- Patrones de diseño en testing
- Análisis de calidad de código
- Debugging de pipelines

DevOps:

- GitHub Actions workflow configuration
- Integración con servicios externos (SonarCloud)
- Manejo de secrets y variables de entorno
- Automatización de procesos

Buenas Prácticas:

- DRY (Don't Repeat Yourself) con Page Objects
- Separación de concerns en tests
- Versionado semántico en commits
- Documentación completa

11.3 Impacto del Proyecto

Beneficios Inmediatos:

- 🔄 Detección temprana de errores
- 🐛 Reducción de bugs en producción
- 📢 Feedback rápido en desarrollo
- 🛠️ Código más mantenible

Beneficios a Largo Plazo:

- 🔄 Mejora continua de calidad
- 🛡️ Despliegues más seguros
- 👥 Mejor colaboración en equipo
- 📊 Métricas de calidad medibles

11.4 Aprendizajes Clave

1. **La configuración importa:** Un error de versión de Node.js puede bloquear todo el pipeline.
2. **Tests E2E son complejos:** Requieren configuración específica para CI/CD.
3. **Automatización ahorra tiempo:** Lo que toma horas manualmente, toma minutos automatizado.
4. **Calidad medible:** SonarCloud proporciona métricas objetivas de calidad.
5. **Debugging es aprendizaje:** Cada error solucionado es conocimiento ganado.

11.5 Recomendaciones Futuras

Para el Proyecto:

1. Solucionar los tests E2E en CI
2. Aumentar cobertura de tests unitarios
3. Configurar Quality Gate más estricto en SonarCloud
4. Implementar deployment automático
5. Agregar tests de rendimiento

Para Otros Desarrolladores:

1. Empezar con tests unitarios simples
 2. Usar Page Objects desde el inicio
 3. Configurar CI/CD temprano en el proyecto
 4. No ignorar los warnings de SonarCloud
 5. Documentar cada decisión técnica
-

12. Referencias

12.1 Documentación Oficial

- Cypress: <https://docs.cypress.io>
- GitHub Actions: <https://docs.github.com/actions>
- SonarCloud: <https://docs.sonarcloud.io>
- Vitest: <https://vitest.dev>
- TypeScript: <https://www.typescriptlang.org/docs>

12.2 Recursos del Proyecto

- Repositorio GitHub: <https://github.com/waldooCreator/W12-11-h>
- Proyecto Original: <https://github.com/cypress-io/cypress-realworld-app>
- GitHub Actions Workflows: <https://github.com/waldooCreator/W12-11-h/actions>
- SonarCloud Dashboard: https://sonarcloud.io/dashboard?id=waldooCreator_W12-11-h

12.3 Herramientas Utilizadas

- Node.js: <https://nodejs.org>
- Yarn: <https://yarnpkg.com>
- Visual Studio Code: <https://code.visualstudio.com>
- Git: <https://git-scm.com>

12.4 Artículos y Guías

- Page Object Pattern: <https://martinfowler.com/bliki/PageObject.html>
 - CI/CD Best Practices: <https://www.atlassian.com/continuous-delivery>
 - Test Automation Pyramid: <https://martinfowler.com/articles/practical-test-pyramid.html>
-

Anexos

Anexo A: Estructura del Proyecto

```
W12-11-h/
├── .github/
│   └── workflows/
│       └── ci.yml                # Pipeline de CI/CD
├── cypress/
│   ├── pages/
│   │   ├── LoginPage.js         # Page Object implementado
│   │   └── index.js
│   ├── tests/
│   │   └── ui/
│   │       └── custom-flow.spec.ts # Tests E2E personalizados
│   └── support/
├── src/
│   ├── __tests__/               # Tests unitarios
│   ├── components/
│   ├── containers/
│   └── models/
├── backend/
├── sonar-project.properties     # Configuración SonarCloud
├── package.json
├── cypress.config.ts
└── README.md
```

Anexo B: Comandos Útiles

```
# Instalación
yarn install

# Tests
yarn test:unit          # Tests unitarios
yarn test:unit:ci       # Tests unitarios en CI
yarn cypress:open       # Cypress UI
yarn cypress:run        # Cypress headless

# Calidad de código
yarn types              # Verificar TypeScript
yarn lint               # Ejecutar ESLint
yarn prettier           # Formatear código

# Desarrollo
yarn dev                # Iniciar app en desarrollo
yarn build              # Compilar para producción
```

Anexo C: Configuración de Secrets en GitHub

Para replicar este proyecto:

1. Ve a: Settings → Secrets and variables → Actions
2. Crea: SONAR_TOKEN con el token de SonarCloud
3. GITHUB_TOKEN se crea automáticamente

Fin del Documento

Este documento fue generado como parte del taller de CI/CD con GitHub Actions, Cypress y SonarCloud.

Autor: Walter Esteban Velasco Contreras

Fecha: 12 de Noviembre de 2025

Repositorio: <https://github.com/waldooCreator/W12-11-h>